
Báo cáo về tạo mô hình dịch Việt-Lào

Thành viên báo cáo bao gồm:

Lương Mạnh Linh 22021215

Lê Quang Thắng 22021209

Đặng Thanh Quang 22021134

Tổng quan về báo cáo

Báo cáo nói về quá trình tìm hiểu thông tin để có thể tạo ra model dịch ngôn ngữ từ tiếng Lào sang tiếng Việt của nhóm chúng em. Trong bài báo cáo này chúng em sẽ nói rõ về các phương pháp đã sử dụng cũng như đánh giá về chất lượng mô hình sau khi thực hiện huấn luyện.

1 Đề bài đặt ra

Chủ đề đặt ra là xây dựng mô hình dịch Việt - Lào từ đầu, sử dụng các loại mô hình để đạt kết quả tốt nhất. Với bộ dữ liệu đã cho là VLSP 2023 – Bộ dữ liệu dịch máy Việt-Lào. Thông tin của bộ dữ liệu VLSP 2023 bao gồm:

- Tập huấn luyện (train): 100.000 cặp câu song ngữ
- Tập phát triển (dev): 2.000 cặp câu
- Tập kiểm thử (test): 1.000 cặp câu
- Định dạng: Tập văn bản UTF-8, với các cặp câu được căn chỉnh 1-1 giữa tiếng Việt và tiếng Lào

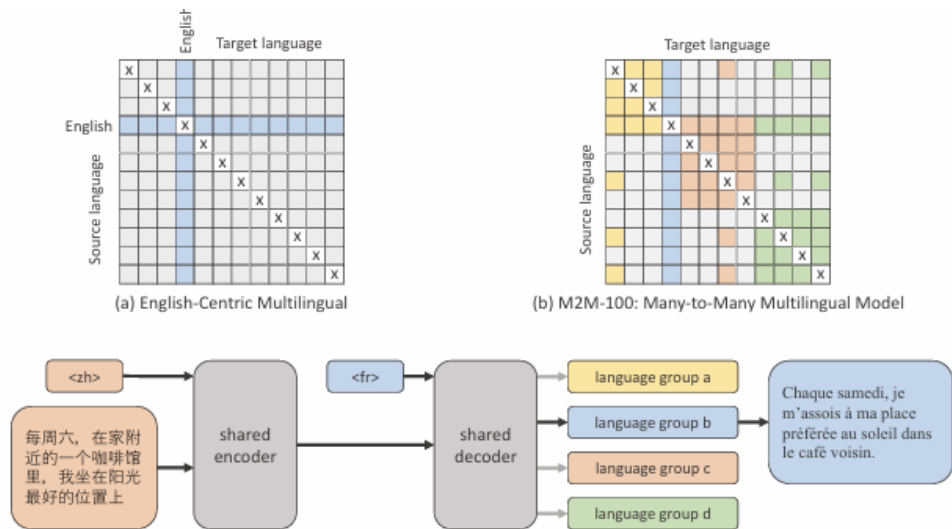
Với base mô hình transformer có sẵn đã cho ở link sau [đây](#).

2 Phương pháp đề xuất

Có rất nhiều kiến trúc tốt để có thể thực hiện được quá trình huấn luyện mô hình. Với base transformer có sẵn chúng em đã thử cải tiến một số cũng như là tối ưu hóa hiện năng tốt nhất có thể để mô hình có thể chạy với tài nguyên ít nhưng đạt hiệu quả khá tốt. Tuy nhiên để đạt hiệu quả cao như kì vọng thì vẫn chưa tốt nên bọn em đã đề xuất sử dụng mô hình M2M100 để có thể training và tạo model tốt hơn.

Về tổng quan model M2M100

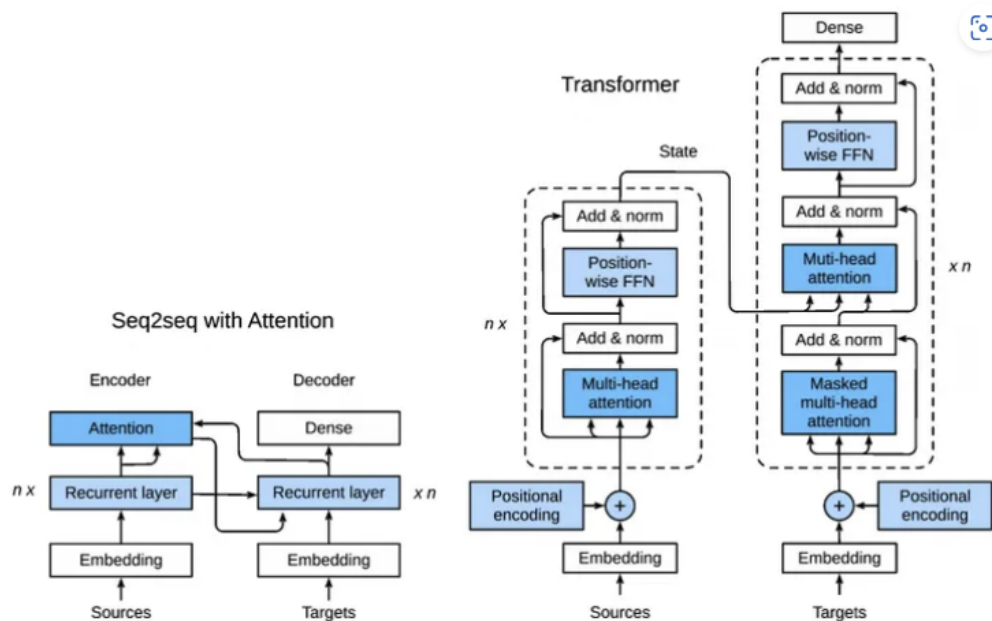
Model M2M100 (Multilingual Machine Translation) là một mô hình dựa trên kiến trúc transformer được phát triển bởi Facebook AI Research (FAIR) trong bài báo Beyond English-Centric Multilingual Machine Translation. Mô hình được thiết kế để dịch trực tiếp giữa tất cả các cặp ngôn ngữ mà không cần phải qua ngôn ngữ trung gian.



Hình 1: Cách hoạt động dịch ngôn ngữ của mô hình

Mô hình sử dụng transformer tương tự như các mô hình BERT hoặc GPT với sơ lược như sau:

- Encoder-Decoder: M2M-100 sử dụng cấu trúc Encoder-Decoder trong Transformer, với phần Encoder sẽ mã hóa câu đầu vào và phần Decoder sẽ tạo ra câu đầu ra.
- Self-attention: Cả Encoder và Decoder đều sử dụng cơ chế self-attention để học các mối quan hệ giữa các từ trong câu, bất kể chúng cách nhau bao xa.
- Cross-attention: Trong phần Decoder, mô hình sử dụng cross-attention để tập trung vào các phần của câu nguồn khi tạo ra câu đích.
- Embedding layer: Các từ được chuyển thành vector thông qua lớp nhúng (embedding) để mô hình có thể xử lý dưới dạng số.



Hình 2: Kiến trúc Transformer trong M2M100

Với một số thành phần chính:

- Encoder là thành phần chịu trách nhiệm mã hóa câu đầu vào (source sentence) thành một biểu diễn trung gian (contextualized representation) trong không gian vector. Trong bối cảnh MMT không tập trung vào tiếng Anh, encoder được thiết kế để xử lý nhiều ngôn ngữ khác nhau một cách thống nhất. Cụ thể:
 - Xử lý đa ngôn ngữ: Encoder nhận đầu vào là câu từ bất kỳ ngôn ngữ nào (ví dụ: tiếng Nhật, tiếng Tây Ban Nha) và chuyển đổi nó thành một tập hợp các vector biểu diễn ngữ nghĩa, độc lập với ngôn ngữ. Để hỗ trợ các cặp ngôn ngữ không phải tiếng Anh, encoder thường được huấn luyện trên dữ liệu đa ngôn ngữ, sử dụng các kỹ thuật như chia sẻ tham số (parameter sharing) hoặc thêm mã ngôn ngữ (language tokens) để phân biệt ngôn ngữ đầu vào.
 - Biểu diễn ngữ cảnh: Encoder sử dụng các tầng Transformer (gồm cơ chế tự chú ý - self-attention và mạng nơ-ron truyền thẳng - feed-forward neural networks) để nắm bắt mối quan hệ ngữ nghĩa và ngữ pháp trong câu đầu vào.
 - Khả năng tổng quát hóa: Trong các mô hình MMT không tập trung vào tiếng Anh, encoder được tối ưu hóa để tạo ra các biểu diễn phù hợp cho các ngôn ngữ có cấu trúc ngữ pháp và từ vựng đa dạng (typologically diverse languages), giảm thiểu thiên kiến hướng về tiếng Anh.
- Decoder chịu trách nhiệm sinh ra câu đầu ra (target sentence) dựa trên biểu diễn trung gian từ encoder. Trong bối cảnh bài báo, decoder được thiết kế để hỗ trợ dịch trực tiếp giữa các ngôn ngữ không phải tiếng Anh và cải thiện hiệu suất zero-shot translation. Cụ thể:
 - Sinh câu đầu ra: Decoder lấy biểu diễn từ encoder và sinh từng từ trong ngôn ngữ mục tiêu (ví dụ: tiếng Nga, tiếng Pháp) theo thứ tự. Nó sử dụng cơ chế chú ý (attention) để tập trung vào các phần liên quan của câu đầu vào, đảm bảo câu đầu ra chính xác về ngữ nghĩa và ngữ pháp.
 - Xử lý đa ngôn ngữ: Tương tự encoder, decoder được huấn luyện để xử lý nhiều ngôn ngữ mục tiêu. Nó có thể sinh câu trong bất kỳ ngôn ngữ nào được hỗ trợ, kể cả khi cặp ngôn ngữ (ví dụ: Nhật → Nga) chưa được huấn luyện trực tiếp, nhờ vào khả năng tổng quát hóa của mô hình.
 - Cải thiện dịch trực tiếp: Trong các phương pháp được đề xuất, decoder được tối ưu hóa để tránh phụ thuộc vào tiếng Anh như ngôn ngữ trung gian, giảm lỗi lan truyền và cải thiện chất lượng dịch cho các cặp ngôn ngữ ít tài nguyên.

3 Quá trình thực hiện

3.1 Sử dụng M2M100

Chuẩn bị bộ dữ liệu ở **đây** cụ thể như sau:

- Sử dụng bộ dữ liệu được cung cấp từ trước VLSP gồm 100.000 bộ train
- Sử dụng Google Translate tự động dịch 100.000 cặp câu từ tiếng Việt sang Lào
- Sử dụng Gemini để tự động tạo ra những cặp câu bao gồm hơn 77.000 cặp câu từ tiếng Việt sang Lào và hơn 54.000 cặp câu từ Lào sang Việt.

Tokenizer và model được sử dụng: **M2M100Tokenizer** và **M2M100ForConditionalGeneration**.

Đặc điểm của **M2M100Tokenizer**: Sử dụng vocabulary được chia sẻ giữa các ngôn ngữ (shared vocabulary), constructor nhận inputs gồm file *json* chứa vocab và file *model* của SentencePiece. SentencePiece được sử dụng nhằm các mục đích sau:

- Chia văn bản thành các đơn vị nhỏ hơn từ (subword units) thay vì sử dụng từ hoặc ký tự nguyên vẹn. Điều này giúp cân bằng giữa độ bao phủ từ vựng và kích thước từ điển, thích hợp cho nhiều hệ ngôn ngữ ở Châu Á.
- Đối với tiếng Lào, do không có khoảng cách tự nhiên giữa các từ nên việc sử dụng SentencePiece giúp mô hình xử lý văn bản hiệu quả hơn. Tương tự, với tiếng Việt, việc chia từ thành các subword giúp mô hình hiểu rõ hơn cấu trúc từ, đặc biệt khi một *từ* có thể bao gồm nhiều *tiếng*.

- Khả năng tùy chỉnh cao đồng thời khá dễ tích hợp trong nhiều model hiện đại.

Đặc điểm của **M2M100ForConditionalGeneration**: có thể dịch Many-To-Many, không cần dịch qua trung gian là tiếng Anh như một số mô hình cũ. Kiến trúc Encoder-Decoder giúp tối ưu hóa việc hiểu và sinh. Dù model đã có phiên bản pretrained, model nhóm sử dụng được tạo từ M2M100Config nhằm mục đích giải quyết đúng bài toán là huấn luyện từ đầu và nhằm giảm kích thước của mô hình.

Dưới đây là tổng quát quá trình thực tế việc huấn luyện và đánh giá M2M100:

1. **Chuẩn bị dữ liệu**: Dữ liệu trong bộ VLSP2023 được sử dụng 100%, được tăng cường bởi dữ liệu cho back-translation và forward-translation sử dụng Google Dịch và Gemini Prompting.
2. **Train sentencepiece model**: Sử dụng dữ liệu trong tập Train của VLSP2023 để xây dựng vocab và sentencepiece model.
3. **Tạo tokenizer và model**: Tokenizer được tạo từ vocab và sentencepiece model, Model được tạo từ Config và không sử dụng pretrained.
4. **Train model**: Sử dụng Seq2SeqTrainer của HuggingFace, thiết lập các thông số trong quá trình train sao cho thích hợp với thiết bị sử dụng và kích thước của model.
5. **Evaluate model**: Sử dụng model có loss thấp nhất trong quá trình train, đánh giá dựa trên bộ dữ liệu Test của VLSP2023, tính BLEU Score và CHRF Score từ hai thư viện là **sacrebleu** và **evaluate** của HuggingFace.

Nhóm thử nghiệm M2M100 cho dịch một chiều và dịch đa ngôn ngữ. Dưới đây là chi tiết của quá trình:

3.1.1 Dịch một chiều

Nhóm bắt đầu với mô hình với mô hình có kích thước nhỏ (khoảng 60M parameters), tạo vocab với size là 24,000, chỉ train cho tác vụ dịch Việt-Lào, dữ liệu train chỉ có tập Train của VLSP2023 và finetune trên tập dữ liệu của từ Google Translate và Gemini. Tuy nhiên, kết quả chỉ đạt BLEU Score trung bình dưới 20.

Tiếp đó, nhóm sử dụng mô hình lớn hơn (khoảng 100M parameters), vocab size được tăng lên 32,000, phương pháp train và finetune cũng tương tự như train. Kết quả BLEU Score trung bình đã tăng lên 26 điểm.

Nhóm tiếp tục tăng kích thước model lên 209M parameters với vocab size 40,000, tạo hai models cho hai tác vụ là dịch Việt-Lào và Lào-Việt. Lần này, vì kích thước model tăng lên, nhóm đã thay đổi dữ liệu train thành VLSP2023 Train + Google Translate và finetune trên dữ liệu Gemini. Kết quả đáng kinh ngạc khi BLEU Score của model dịch Lào-Việt tăng lên 41 với cả sacrebleu và evaluate, BLEU Score của model dịch Việt-Lào đạt 56 (một con số vô lý) với sacrebleu và 29 với evaluate. Dưới đây là cấu hình của model 209M:

- **Số lớp Encoder**: 8
- **Số lớp Decoder**: 8
- **Kích thước vector biểu diễn (d_model)**: 1024
- **Số đầu chú ý (attention heads)**: 8
- **Kích thước feedforward network**: 2048
- **Hàm kích hoạt**: GELU
- **Dropout**: 0.3
- **Tổng số tham số**: 209,068,032

Kết quả đánh giá mô hình trên tập Test của VLSP2023:

Mô hình	BLEU (Sacrebleu)	BLEU (Evaluate)	CHRF (Sacrebleu)	CHRF (Evaluate)
Thang điểm	0:100	0:1	0:10	0:100
Lào-Việt	41.41	0.4188	4.97	53.29
Việt-Lào	56.55	0.2874	4.60	52.56

Cả hai mô hình hiện đã có trên HuggingFace, có thể được sử dụng tại [đây](#).

3.1.2 Dịch đa ngôn ngữ

Nhóm đã cố gắng tạo ra một mô hình có thể dịch được cả Việt-Lào và Lào-Việt, bởi M2M100 được sinh ra là để thực hiện công việc này. Có hai cách đã được thực hiện để tạo ra một Multilingual M2M100:

- Train từ đầu
- Finetune một model ở phần dịch một chiều

Tuy nhiên, kết quả nhận được từ cả hai cách trên đều rất thấp nên nhóm sẽ không sử dụng mô hình này.

3.1.3 Quá trình huấn luyện

Dưới đây là các thông số trong quá trình huấn luyện *Model M2M100 209M*.

- **Tối ưu hóa:** AdamW với weight decay 0.01
- **Tốc độ học (learning rate):** 1e-3
- **Số epochs:** 15 với early stopping (patience=5)
- **Loss function:** Cross Entropy với padding_idx
- Sử dụng **torch_compile** và **fp16** nhằm tăng tốc độ huấn luyện
- Sử dụng **max_grad_norm=0.5** để tránh hiện tượng **exploding gradients**

Epoch	Train Loss	Val. Loss
1	0.875700	1.072054
5	0.393600	0.521319
10	0.253100	0.395378
13	0.221800	0.370641
15	0.208500	0.367189

Hình 3: Diễn biến huấn luyện mô hình M2M100 209M Việt-Lào

3.1.4 Tài nguyên sử dụng

Trong phần này, nhóm sẽ thống kê tài nguyên sử dụng cho *Model M2M100 200M*.

Quá trình train:

- **GPU:** A100
- **Epochs:** 15
- **Train data:** 200,000 cặp câu Việt-Lào
- **Thời gian:** 01 giờ 35 phút

Quá trình finetune:

- **GPU:** P100
- **Epochs:** 3
- **Train data:** 200,000 cặp câu Việt-Lào
- **Thời gian:** 04 giờ 05 phút

Quá trình dịch một câu:

- **GPU:** P100
- **Thời gian trung bình:** 2.8 giây

3.2 Đánh giá mô hình

Mặc dù đạt điểm BLEU cao, nhưng *Model M2M100 209M* lại cần rất nhiều tài nguyên cho quá trình huấn luyện và suy luận. Với lượng dữ liệu không quá lớn và phải train từ đầu, sử dụng một model có kích thước lớn như vậy có thể sẽ không phù hợp và gây lãng phí. Vậy nên việc sử dụng một mô hình có kích thước nhỏ hơn như trình bày dưới đây có thể được xem xét.

3.3 Transformer tiêu chuẩn

Mô hình dịch máy Neural Lào-Việt được xây dựng dựa trên kiến trúc Transformer với các cải tiến phù hợp cho cặp ngôn ngữ ít tài nguyên này. Mô hình đã đạt được điểm **BLEU 37.47** trên tập kiểm thử, một kết quả đáng khích lệ trong bối cảnh dữ liệu song ngữ còn hạn chế.

3.3.1 Kiến trúc mô hình

Mô hình được thiết kế dựa trên kiến trúc Transformer chuẩn với các tham số được tối ưu hóa để phù hợp với tài nguyên tính toán có sẵn:

- Số lớp Encoder: 6
- Số lớp Decoder: 6
- Kích thước vector biểu diễn (**d_model**): 512
- Số đầu chú ý (**attention heads**): 8
- Kích thước **feedforward network**: 1024
- Hàm kích hoạt: GELU (thay vì ReLU truyền thống)
- Tỷ lệ Dropout: 0.1
- Tổng số tham số có thể huấn luyện: 56,137,344

3.3.2 Tokenization và xử lý dữ liệu

Chúng em đã áp dụng một số kỹ thuật tiền xử lý đặc biệt để xử lý hiệu quả cặp ngôn ngữ Lào-Việt:

- **SentencePiece Tokenizer**: Sử dụng phương pháp BPE (Byte Pair Encoding) với `vocab_size = 16.000` cho mỗi ngôn ngữ. Phương pháp này đặc biệt phù hợp với tiếng Lào vì đây là ngôn ngữ không có khoảng trắng tự nhiên giữa các từ.
- **Tăng cường dữ liệu**: Áp dụng kỹ thuật phân tách câu thông minh, cụ thể như sau:
 - Các câu dài (>15 từ ở cả hai bên) được tách thành các câu ngắn hơn
 - Chỉ giữ lại các cặp câu có độ dài tương đương ở cả hai ngôn ngữ
 - Loại bỏ các câu quá ngắn (<3 từ) sau khi tách
- **Xử lý đặc biệt cho tiếng Lào**: Phát triển tokenizer tiếng Lào đặc thù với coverage cao (0.9995) để đảm bảo xử lý tốt các ký tự đặc biệt.

3.3.3 Tối ưu hóa hiệu năng

Để mô hình có thể hoạt động hiệu quả trong môi trường có giới hạn tài nguyên (Google Colab với GPU Tesla T4), chúng em đã áp dụng nhiều kỹ thuật tối ưu hóa:

- **Huấn luyện độ chính xác hỗn hợp (Mixed Precision)**: Sử dụng FP16/FP32 để giảm lượng bộ nhớ GPU cần thiết và tăng tốc quá trình huấn luyện.
- **Gradient Checkpointing**: Kỹ thuật này cho phép tiết kiệm bộ nhớ trong quá trình lan truyền ngược bằng cách tính toán lại một số activations thay vì lưu trữ tất cả, giúp huấn luyện mô hình lớn hơn với cùng dung lượng bộ nhớ.
- **Tích lũy gradient (Gradient Accumulation)**: Mô phỏng batch size lớn hơn bằng cách tích lũy gradient qua 4 mini-batch trước khi cập nhật tham số.

- **Pre-Layer Normalization:** Sử dụng cấu trúc Pre-LN thay vì Post-LN trong các khối Transformer để tăng tính ổn định trong quá trình huấn luyện.
- **Khởi tạo tham số thông minh:** Sử dụng phương pháp khởi tạo Xavier để cải thiện quá trình hội tụ.

3.3.4 Quá trình huấn luyện

Mô hình được huấn luyện với cấu hình tối ưu cho cân bằng giữa chất lượng và hiệu quả tài nguyên:

- **Tối ưu hóa:** AdamW với weight decay 0.01
- **Tốc độ học (learning rate):** 1e-3 với lịch trình OneCycleLR
- **Batch size thực tế:** 16 (64 sau khi tích lũy gradient)
- **Thời gian huấn luyện:** Mỗi epoch khoảng 8 phút
- **Số epochs:** 15 với early stopping (patience=5)
- **Loss function:** Cross Entropy với padding_idx

Epoch	Train Loss	Val. Loss	BLEU Score
1	7.6772	7.1115	0.84
5	4.0652	3.9296	8.90
10	2.2396	2.9944	14.34
13	1.7261	2.9238	28.01
15	1.4820	2.9534	23.18

Hình 4: Diễn biến huấn luyện mô hình Transformer Lào-Việt

Như có thể thấy trong Bảng 3, mô hình đạt điểm BLEU tốt nhất (28.01) tại epoch 13, sau đó bắt đầu có dấu hiệu overfitting khi điểm BLEU giảm xuống 23.18 ở epoch 15.

3.3.5 Đánh giá mô hình

Mô hình đạt điểm BLEU 37.47 trên tập kiểm thử. Đây là kết quả tốt với một mô hình Transformer có kích thước trung bình, đặc biệt khi xét đến việc dịch giữa cặp ngôn ngữ ít tài nguyên như Lào-Việt.

Quá trình training: Sử dụng GPU T4 của Google Colab thực hiện training tạo ra model, kết quả model tạo ra ở [đây](#)

Đoạn code thực hiện ở [đây](#)

4 Đánh giá và kết luận

Việc đánh giá được thực hiện trên tập Test với 1,000 cặp câu Việt-Lào, với các thông số như BLEU và CHRF. Trong quá trình đánh giá, câu đã được tokenize bằng các thư viện như **pyvi** và **laonlp** trước khi *join* lại để tính toán các điểm số trên.

Kết quả đánh giá rất khách quan với hai phương pháp được đề xuất:

- Đối với M2M100, BLEU đạt 41 đối với dịch Lào-Việt, đạt 56 (sacrebleu) và 29 (evaluate) đối với dịch Việt-Lào.
- Đối với transformer tiêu chuẩn, mô hình đạt điểm khá rất tốt khi dùng sacrebleu lên tới 37.47 trong dịch Lào-Việt.

Kết luận lại, khi tài nguyên tính toán cho phép, ta có thể sử dụng M2M100 vì độ tin cậy và sự hoạt động mượt mà trong tác vụ dịch máy. Ngược lại, sử dụng Transformer tiêu chuẩn cũng là một lựa chọn hợp lý khi kết quả đem lại rất xứng đáng cho kích thước khiêm tốn của nó.

5 Tổng kết đánh giá thành viên

Bọn em phân ra tìm hiểu và thực hiện train các model và lần lượt đánh giá kết quả của nhau tron 3 tuần đầu. Sau 3 tuần đầu bọn em tổng kết lại lựa chọn model của bạn phát triển nhất để tập trung trọng điểm và phát triển thêm. Về đóng góp của nhóm em như sau:

- Lương Mạnh Linh: sử dụng mô hình M2M100 là kết quả được đề xuất trong báo cáo của chúng em, đạt được số điểm khá là ấn tượng như trên
- Đặng Thanh Quang: Bạn sử dụng kiến trúc transformer chuẩn với điều chỉnh lại hiệu năng tối ưu cách tốt nhất: Tổng quan về mô hình bạn sử dụng
 - Việc sử dụng tokenizer: vẫn sử dụng kiến trúc sentencepiece để tạo tokenizer cho mô hình
 - Xử lý dữ liệu đầu vào: augment-data tăng việc sử dụng song ngữ đồng thời tách những câu có cấu trúc dài để thuận tiện trong việc huấn luyện
 - Về kiến trúc transformer: sử dụng kiến trúc transformer tiêu chuẩn và sử dụng gradient checkpointing để có thể giảm bộ nhớ để phù hợp với môi trường training bằng colab với gpu T4.
 - Model đạt hiệu quả tốt với bộ tính điểm sacrebleu với 37.47 điểm
- Lê Quang Thắng: Sử dụng kiến trúc transformer chuẩn điều chỉnh lại bằng cách sử dụng ROPE và Sampling nhưng đạt hiệu quả không tốt đồng thời tốn khá nhiều tài nguyên hơn bạn Quang, điểm khi chấm ở bộ điểm sacrebleu chỉ đạt ở mức 23.64 điểm.